

HOJA DE TRABAJO — SEMANA 1 — DD281

PROGRAMACIÓN SEGURA

Introducción a la Programación Segura y Modelado de Seguridad

🕒 5 Horas Académicas Presenciales

Docente	RUBEN AVISPE LACCAWIMAY	Semana	Semana 1 · Semestre 202601
Estudiante	LINDO BERNARDOS J Kohn		
Fecha	07/06/2026	Nota Final	

🎯 Resultado de Aprendizaje

Al finalizar la sesión, el estudiante elabora un plan de desarrollo y diseña casos de uso relacionados con la seguridad del sistema.

Sec.	Peso	Contenido	Tiempo
A	15%	Completar la Frase — 20 ejercicios de conceptos clave	~30 min
B	15%	Relacionar Conceptos — 20 pares columna A / columna B	~25 min
C	35%	Completar el Código — 15 ejercicios en Java	~70 min
D	20%	Análisis de Código — 10 fragmentos seguros e inseguros	~45 min
E	—	Modelado de Amenazas — Caso RetailPlus (5 tablas)	~35 min
F	—	Casos de Uso Seguros — 3 plantillas completas	~25 min
G	—	Plan de Desarrollo Seguro — Tabla SDLC 6 fases	~10 min
H	20%	Mini Reto de PC — Login seguro en Java	~30 min

⚠️ Desarrolle toda la guía A MANO. No se aceptan respuestas impresas. Cuide la ortografía y la claridad de sus diagramas.

A

COMPLETE LA FRASE

20 ejercicios — escriba el término técnico más preciso

15%

-30 min

Complete cada oración con el concepto de programación segura correspondiente. Escriba el término con letra clara sobre la línea de respuesta.

1. La AUTENTICACIÓN es el proceso de verificar la identidad de un usuario antes de permitirle el acceso a un sistema.

Respuesta: AUTENTICACIÓN

2. La _____ garantiza que la información no sea modificada sin autorización por parte de personas no autorizadas.

Respuesta: INTEGRIDAD

3. La _____ asegura que los sistemas y datos estén disponibles para los usuarios autorizados en el momento en que los necesitan.

Respuesta: DISPONIBILIDAD

4. Una _____ es una debilidad en el diseño, implementación o configuración de un sistema que puede ser explotada por un atacante.

Respuesta: VULNERABILIDAD

5. Una _____ es cualquier circunstancia o evento con el potencial de causar daño a un activo de información.

Respuesta: AMENAZA

6. El _____ se define como la combinación de la probabilidad de que ocurra un evento adverso y el impacto resultante sobre los activos.

Respuesta: RIESGO

7. OWASP son las siglas de _____, una fundación sin fines de lucro que publica el Top 10 de vulnerabilidades web.

Respuesta: OPEN WEB APPLICATION SECURITY PROJECT

8. La _____ define qué recursos o acciones puede ejecutar un usuario dentro del sistema una vez que ya ha sido autenticado.

Respuesta: AUTORIZACIÓN

9. Un _____ es cualquier recurso de valor para la organización, como datos, sistemas, infraestructura o personas.

Respuesta: ACTIVO

10. El principio de _____ establece que cada usuario debe tener únicamente los permisos necesarios para realizar su trabajo.

Respuesta: MENOR PRIVILEGIO

11. La _____ de un sistema consiste en definir los requerimientos de seguridad desde las fases iniciales del desarrollo del software.

Respuesta: SEGURIDAD POR DISEÑO

12. Un _____ es un mecanismo, proceso o tecnología implementado para reducir la probabilidad o el impacto de una amenaza o vulnerabilidad.

Respuesta: CONTROL DE SEGURIDAD

13. La _____ es una técnica de ataque que manipula a las personas para obtener información confidencial aprovechando su confianza.

Respuesta: INGENIERIA SOCIAL

14. El _____ garantiza que las acciones realizadas en un sistema puedan atribuirse de forma inequívoca e irrefutable a quien las ejecutó.

Respuesta: NO REPUDIO

15. El _____ de amenazas es una técnica sistemática para identificar y priorizar posibles ataques contra un sistema antes de construirlo.

Respuesta: MODELADO

16. La criptografía _____ utiliza un par de claves —una pública y una privada— para cifrar y descifrar información de forma segura.

Respuesta: ASIMETRICA

17. Una función de _____ transforma datos de cualquier tamaño en una cadena de longitud fija, siendo un proceso matemáticamente irreversible.

Respuesta: HASH

18. En los _____ seguros se documenta el flujo que el sistema debe seguir para proteger una funcionalidad frente a posibles ataques o abusos.

Respuesta: CASO DE USO

19. La autenticación _____ (MFA) combina dos o más factores de verificación para aumentar significativamente la seguridad del acceso.

Respuesta: MULTI FACTOR

20. El ciclo de vida de desarrollo seguro (_____) integra actividades y controles de seguridad en cada fase del proceso de construcción del software.

Respuesta: SSDLC
SECURE SOFTWARE DEVELOPMENT LIFE CYCLE

B

RELACIONE CONCEPTOS

Escriba en la columna central la letra de la definición correspondiente

15%

~25 min

Columna A — Concepto		Columna B — Definición
1. Amenaza	S	a) Software malicioso diseñado para dañar, robar o comprometer sistemas.
2. Vulnerabilidad	S	b) Proceso de verificar que un usuario es quien dice ser.
3. Riesgo	M	c) Técnica que inserta código malicioso en páginas web para ejecutarse en el navegador del visitante.
4. Firewall	F	d) Recurso de valor para la organización: datos, hardware, software o servicios.
5. IDS	h	e) Fundación que publica el Top 10 de vulnerabilidades web más críticas.
6. IPS	k	f) Mecanismo que filtra el tráfico de red según reglas de seguridad predefinidas.
7. OWASP	C	g) Acción deliberada que explota una vulnerabilidad para comprometer la seguridad.
8. Hash	N	h) Sistema que detecta actividad sospechosa en la red y genera alertas.
9. Cifrado	i	i) Proceso de transformar datos legibles en un formato ilegible mediante una clave.
10. MFA	q	j) Debilidad en un sistema que puede ser aprovechada por un atacante.
11. Autenticación	b	k) Sistema que detecta Y bloquea activamente el tráfico malicioso en la red.
12. Autorización	P	l) Técnica que manipula a personas para revelar información confidencial.
13. Activo	d	m) Combinación de probabilidad de ataque e impacto sobre los activos.
14. Control de seguridad	r	n) Función que transforma datos en una cadena de longitud fija e irreversible.
15. Ataque	g	o) Técnica sistemática para identificar amenazas en un sistema antes de construirlo.
16. Ingeniería social	L	p) Mecanismo que define qué puede hacer un usuario ya autenticado.
17. Malware	a	q) Uso de dos o más factores para verificar la identidad de un usuario.
18. SQL Injection	T	r) Medida implementada para reducir la probabilidad o impacto de una amenaza.
19. XSS	C	s) Evento o circunstancia con potencial de causar daño a un activo.
20. Modelado de amenazas	O	t) Ataque que inyecta sentencias SQL en formularios para manipular la base de datos.

C

COMPLETE EL CÓDIGO

15 ejercicios en Java — complete los espacios en blanco con la instrucción correcta

35%

~70 min

Ejercicio 1: Validación de longitud mínima de contraseña

```
Scanner sc = new Scanner(System.in);
System.out.print("Ingrese contraseña: ");
String password = sc.nextLine();

if (password.length() < 8) {
    System.out.println("Error: contraseña demasiado corta");
} else {
    System.out.println("Longitud aceptable");
}
```

💡 Pista: Longitud mínima recomendada por NIST para contraseñas: 8 caracteres.

Ejercicio 2: Verificar presencia de dígito en la contraseña

```
boolean tieneNumero = false;
for (char c : password.toCharArray()) {
    if (Character.isDigit(c)) {
        tieneNumero = true;
        break;
    }
}

System.out.println("¿Tiene número? " + tieneNumero);
```

💡 Pista: Método de la clase Character que retorna true si el carácter es un dígito (0-9).

Ejercicio 3: Verificar presencia de letra mayúscula

```
boolean tieneMayuscula = false;
for (char c : password.toCharArray()) {
    if (Character.isUpperCase(c)) {
        tieneMayuscula = true;
        break;
    }
}

if (!tieneMayuscula)
    System.out.println("Incluya al menos una mayúscula");
```

💡 Pista: Método de Character que retorna true si el carácter es una letra mayúscula.

Ejercicio 4: Validación completa de contraseña robusta

```
public static boolean esContraseñaSegura(String pass) {
    if (pass == null || pass.Length < 8) return false;
    boolean num = pass.MATCHES("^[0-9]*[0-9].*");
    boolean may = pass.MATCHES("^[A-Z]*[A-Z].*");
```



```

boolean sim = pass.MATCHES(".*[!@#$$%^&*].*");
return num && may && sim;
}

```

 **Pista:** Propiedad de longitud de String — Método que evalúa si una cadena cumple una expresión regular.

Ejercicio 5: Validación de correo electrónico con expresión regular

```

import java.util.regex.*;

String correo = sc.nextLine();
Pattern patron = Pattern.compile(
    "^[\\w._-]+@[\\w.-]+\\.\\w+$");
Matcher m = patron.matcher(correo);

if (m.MATCHES()) {
    System.out.println("Correo válido");
} else {
    System.out.println("Correo inválido");
}

```

 **Pista:** Patrón: `^[\\w._-]+@[\\w.-]+\\.\\w+$` — Método de Matcher que verifica coincidencia total.

Ejercicio 6: Validación de nombre de usuario alfanumérico

```
String usuario = sc.nextLine().trim();

if (!usuario.matches("[a-zA-Z0-9_]{4,20}$")) {
    System.out.println("Usuario inválido: solo letras, " +
        "números y guion bajo, entre 4 y 20 caracteres");
}
```

💡 Pista: Método que elimina espacios al inicio y al fin — Método que evalúa la cadena contra una expresión regular.

Ejercicio 7: Sanitización básica: eliminar caracteres peligrosos

```
String entrada = sc.nextLine();
// Elimina < > ' " ; -- para prevenir inyecciones básicas
String limpia = entrada
    .replace("<", "").replace(">", "")
    .replace("'", "").replace("\"", "")
    .replace(";", "").replace("--", "");
System.out.println("Entrada sanitizada: " + limpia);
```

💡 Pista: Método de String que reemplaza una subcadena por otra (no es replace con char, es con String).

Ejercicio 8: Control de acceso basado en roles con switch

```
String rol = "VENDEDOR";

switch (rol) {
    case "ADMIN":
        System.out.println("Acceso total al sistema");
        break;
    case "VENDEDOR":
        System.out.println("Acceso a ventas y carrito");
        break;
    default:
        System.out.println("Rol no reconocido: acceso denegado");
}
```

💡 Pista: Estructura de control múltiple — Instrucción para salir del bloque — Palabra clave del caso por defecto.

Ejercicio 9: Contador de intentos fallidos — bloqueo de cuenta

```
int intentosFallidos = 0;
final int LIMITE = 3;
boolean bloqueado = false;

while (intentosFallidos < LIMITE && !bloqueado) {
    String pass = sc.nextLine();
    if (pass.equals("ClaveCorrecta")) {
        System.out.println("Acceso concedido");
    }
}
```



```

        break;
    } else {
        intentosFallidos++;
        if (intentosFallidos >= LIMITE) {
            bloqueado = True;
            System.out.println("Cuenta bloqueada temporalmente");
        }
    }
}

```

 **Pista:** Límite recomendado: 3 — Método de comparación de cadenas sensible a mayúsculas — Valor booleano.

Ejercicio 10: Validar que la contraseña no sea igual ni contenga el nombre de usuario

```

String usuario = sc.nextLine().toLowerCase();
String password = sc.nextLine().toLowerCase();

if (password.EQUALS (usuario)) {
    System.out.println("La contraseña no puede ser igual al usuario");
} else if (password.CONTAINS (usuario)) {
    System.out.println("La contraseña no puede contener el usuario");
} else {
    System.out.println("Contraseña aceptada");
}

```

 **Pista:** Método de comparación de contenido exacto — Método que verifica si una cadena contiene a otra.

Ejercicio 11: Estructura if-else encadenada para validar datos de registro

```
String nombre = sc.nextLine().trim();
String email = sc.nextLine().trim();
String pass = sc.nextLine();

if (nombre.isEmpty()) {
    System.out.println("El nombre es obligatorio");
} ELSE IF (!email.Matches("^[\\w._%+-]+@[\\w.-]+\\.[a-zA-Z]{2,}$")) {
    System.out.println("Email inválido");
} ELSE IF (pass.length() < 8) {
    System.out.println("Contraseña muy corta");
} else {
    System.out.println("Registro válido");
}
```

💡 Pista: Método que retorna true si la cadena está vacía — else if — matches.

Ejercicio 12: Confirmación de nueva contraseña (verificar que coincidan)

```
System.out.print("Nueva contraseña: ");
String pass1 = sc.nextLine();
System.out.print("Confirme contraseña: ");
String pass2 = sc.nextLine();

if (!pass1.Equals(pass2)) {
    System.out.println("Las contraseñas no coinciden");
    return;
}
if (pass1.length() < 8) {
    System.out.println("Mínimo 8 caracteres requeridos");
    return;
}
System.out.println("Contraseña actualizada correctamente");
```

💡 Pista: Método de comparación de contenido — Longitud mínima segura recomendada.

Ejercicio 13: Validar que el ID de usuario sea un entero positivo

```
System.out.print("ID de usuario: ");
String inputId = sc.nextLine().trim();
int userId = -1;

try {
    userId = Integer.parseInt(inputId);
    if (userId <= 0) {
        System.out.println("ID debe ser positivo");
        userId = -1;
    }
} catch (NumberFormatException) {
}
```



```

        System.out.println("ID no válido: ingrese un número entero");
    }
}

```

💡 Pista: Método para parsear String a int — Operador menor o igual que — Excepción para formato numérico inválido.

Ejercicio 14: Validación de rango en menú de opciones

```

System.out.println("1. Iniciar sesión | 2. Registrarse | 3. Recuperar contraseña");
System.out.print("Seleccione una opción: ");

int opcion = -1;
try {
    opcion = Integer.parseInt(sc.nextLine());
} catch (NumberFormatException e) {
    System.out.println("Opción inválida");
}

if (opcion < 1 || opcion > 3) {
    System.out.println("Opción fuera de rango");
}

```

💡 Pista: Operadores de comparación y operador lógico OR: < || >

Ejercicio 15: Mensaje de error genérico — no revelar si falló el usuario o la contraseña

```

// PRINCIPIO: no indicar cuál de los dos campos es incorrecto
boolean usuarioExiste = buscarUsuario(user);
boolean contrasenaCorrecta = verificarPass(user, pass);

if (!usuarioExiste || !contrasenaCorrecta) {
    // Mensaje AMBIGUO a propósito – nunca especifique cuál falló
    System.out.println("Credenciales inválidas");
} else {
    iniciarSesion(user);
    System.out.println("Bienvenido, " + user);
}

```

💡 Pista: Niegue (!) cada condición — El mensaje debe ser ambiguo intencionalmente (ej: "Credenciales inválidas").

D

ANÁLISIS DE CÓDIGO

10 fragmentos — identifique el problema, explique el riesgo y proponga mejora

20%
-45 min

Algunos fragmentos son seguros y otros inseguros. Analice cada uno con criterio técnico y responda las 3 preguntas.

Fragmento 1 [Contraseña en texto plano] ⚠ Posible problema

```
Scanner sc = new Scanner(System.in);
System.out.print("Contraseña: ");
String pass = sc.nextLine();

if (pass.equals("Admin2024")) {
    System.out.println("Acceso concedido");
} else {
    System.out.println("Acceso denegado");
}
```

1. Identifique el problema de seguridad

La contraseña "Admin2024" está codificada en el código fuente y es almacenada en texto plano con equals()

2. Explique el riesgo que genera

Cualquiera con acceso al código podría ver la clave.

3. Proponga corrección

Almacena el hash de la clave con SAL (bcrypt, Argon2, PBKDF2) en la base de datos y hacer comparaciones.

Fragmento 2 [Validación de entrada] ✓ Aparentemente seguro

```
public static boolean validarUsuario(String usuario) {
    if (usuario == null || usuario.trim().isEmpty()) {
        return false;
    }
    return usuario.trim().matches("^[a-zA-Z0-9_]{4,20}$");
}
```

1. Identifique el problema o confirme que es seguro y por qué

Es seguro: Valida null, cadena vacía y aplica lista blanca con formato y longitud (4-20).

2. Principio de seguridad que aplica

• APLICA VALIDACION DE ENTRADAS POR LISTA BLANCA
• SOLO SE ACEPTAN CARACTERES PERMITIDOS

3. ¿Qué mejoraría o qué le falta?

NORMALIZAR UNICODE ANTES DE VALIDAR Y REUTILIZAR EL MÉTODO EN TODOS LOS PUNTOS DE ENTRADA.

Fragmento 3 [Ausencia total de validación] ⚠ Posible problema


```
Scanner sc = new Scanner(System.in);
System.out.print("Nombre de usuario: ");
String user = sc.nextLine();
System.out.print("Email: ");
String email = sc.nextLine();
// Se usa directamente sin ninguna validación
registrarUsuario(user, email);
```

1. Identifique el problema de seguridad

Las variables user y email se usan en registrarUsuario() sin validación.

2. Explique el riesgo que genera

Puede ser víctima de inyección (SQL/XSS), datos corruptos, credenciales inválidas, almacenamiento de datos corruptos en la BD.

3. Proponga corrección

Validar formato y longitud, usar consultas parametrizadas y codificar la salida antes de que se muestre.

Fragmento 4 [Control de acceso por rol]

✓ Aparentemente seguro

```
String rol = sesion.obtenerRol();
switch (rol) {
    case "ADMIN":    mostrarPanelAdmin();    break;
    case "VENDEDOR": mostrarPanelVentas();    break;
    default:          System.out.println("Acceso denegado");
}
```

1. Confirme si es seguro o identifique el problema

ES CONFIABLE Y FUNCIONAL, INCLUYE DEFAULT QUE DENIEGA EL ACCESO, DEPENDE DEL ROL QUE PROVENGA DEL SERVIDOR.

2. Principio de seguridad aplicado

APLICA EL PRINCIPIO DON DEFECTO Y CONTROL DE ACCESO PASADO EN ROL'S

3. ¿Qué mejoraría?

QUE EL ROL NO SEA NULL, REGISTRAR LOS ACCESO DENEGADOS.

Fragmento 5 [Construcción de consulta SQL]

△ Posible problema

```
String usuario = sc.nextLine();
String consulta = "SELECT * FROM usuarios "
    + "WHERE nombre = '" + usuario + "'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(consulta);
```

1. Identifique el problema de seguridad

LA ENTRADA DEL USUARIO SE CONECTA DIRECTAMENTE DENTRO DE LA SENTENCIA SQL = INYECCION SQL.

2. Explique el riesgo que genera

PUEDE SUFIR UNA INYECCION SQL => CON ' OR '1'='1 PARA EVADIR FILTROS, EXTRAER DATOS DE LA TABLA, MODIFICAR, ELIMINAR LOGIN

3. Proponga corrección con PreparedStatement

USAR CONSULTAS PARAMETRIZADAS. CON PREPAREDSTATEMENT.

"SELECT * FROM USUARIOS WHERE NOMBRE = ?" ps.setString(1, usuario)

Fragmento 6 [Mensaje de error revelador]

△ Posible problema

```
if (!usuarioExiste(user)) {
    System.out.println("El usuario '" + user + "' no existe en el sistema");
} else if (!passwordCorrecta(user, pass)) {
    System.out.println("Contraseña incorrecta para el usuario " + user);
}
```

1. Identifique el problema de seguridad

ENUMERACION DE USUARIOS: LOS DIFERENTES MENSAJES REVELAN SI LA CUENTA EXISTE Y SI LA CONTRASEÑA ES INCORRECTA

△ 2. Explique el riesgo que genera

UN ATAQUE PUEDE ENUMERAR CUENTAS VÁLIDAS, CONCENTRANDO ATAQUES DE FUERZA BRUTA.

🔗 3. Proponga corrección

USAR UN SOLO MENSAJE (¡CONSENCIALES INVÁLIDAS!)

Fragmento 7 [Bloqueo por intentos fallidos]

✓ Aparentemente seguro

```
int intentos = 0; final int MAX = 3;
while (intentos < MAX) {
    String p = sc.nextLine();
    if (verificar(user, p)) { System.out.println("Sesión iniciada"); break; }
    intentos++;
    if (intentos == MAX) System.out.println("Cuenta bloqueada temporalmente");
}
```

🔍 1. Confirme si es seguro o identifique el problema

ES SEGURO, BLOQUEA TRAS MAX(3) INTENTOS, VIVE EN FORMA BUENA

△ 2. Principio de seguridad que aplica

APLICA PROTECCIÓN CONTRA FUERZA BRUTA

🔗 3. ¿Qué mejoraría?

PERSISTIR EL CONTADOR POR CUENTA EN BD PARA QUE EL BLOQUEO SOBREVIVA AL REINICIO, AÑADIR CAPTCHA.

Fragmento 8 [SQL con concatenación] ⚠ Posible problema

```
String cat = sc.nextLine();
String sql = "SELECT nombre, precio FROM productos "
    + "WHERE categoria = '" + cat + "' ORDER BY precio ASC";
ResultSet rs = stmt.executeQuery(sql);
```

🔍 1. Problema · 2. Riesgo · 3. Corrección

1. INYECCIÓN SQL
 2. CONSULTA MANIPULABLE
 3. PARAMETRIZAR CONSULTA PREPARED STATEMENT

Fragmento 9 [Validación robusta de contraseña] ✓ Aparentemente seguro

```
public static boolean esRobusta(String p) {
    return p != null && p.length() >= 8
        && p.matches(".*[A-Z].*") && p.matches(".*[0-9].*")
        && p.matches(".*[!@#$%].*");
}
```

🔍 1. Confirme si es seguro · 2. Principio que aplica · 3. ¿Qué mejoraría?

1. ES SEGURO, VERIFICA LONGITUD MINIMA, MAYUSCULA, DIGITO Y SIMBOLO
 2. APLICA POLITICA DE CONTRASEÑA FUERTE
 3. FIJAN LONGITUD MAXIMA

Fragmento 10 [Rol definido por el propio usuario] ⚠ Posible problema

```
System.out.print("Ingrese su rol: ");
String rol = sc.nextLine(); // El usuario define su propio rol
if (rol.equals("ADMIN")) {
    System.out.println("Acceso al panel administrativo concedido");
    cargarDatosAdministrativos();
}
```

🔍 1. Problema · 2. Riesgo · 3. Corrección

1. SISTEMA CONFIA EN ENTRADA DE ROL DEL USUARIO
 2. CUALQUIERA PUEDE INGRESAR ADMIN Y ACCEDER AL PANEL ADMIN
 3. ASIGNAR ROL EN BD TRAS AUTENTICACION, NO A PARTIR DE USUARIO

La empresa RetailPlus está desarrollando un sistema web de ventas online que incluirá: módulo de login y registro de usuarios, carrito de compras, procesamiento de pagos con tarjeta de crédito y panel de administración para gestión de inventario.

E.1 — Activos Críticos del Sistema

N°	Activo Crítico	Tipo	Criticidad
1	CREDENCIALES DE USUARIO	DATOS	ALTA
2	DATOS DE TARJETA DE CREDITOS	DATOS	CRITICA
3	INFORMACION DE CLIENTES	DATOS	ALTA
4	BD DE TRANSACCIONES / VENTAS	DATOS	ALTA
5	PANEL DE ADMINISTRACION	APLICACION	CRITICA
6	MODULO / PASARELAS DE PAGOS	SERVICIO	CRITICA
7	INV. DE PRODUCTOS	DATOS	MEDIA
8	SERVIDOR WEB / DE APP	INFRAESTRUCTURA	ALTA
9	SESIONES Y TOKENS DE AUTENTICA.	DATOS	ALTA
10	LOGS Y REGISTROS DE AUDITORIA	DATOS	MODER

E.2 — Amenazas del Sistema

N°	Amenaza	Componente afectado	Probabilidad	Impacto
1	inyección SQL	LOGIN / BD	Media	Alto
2	Cross-site (XSS)	Registro / Login	Media	Medio
3	Fuerza Bruta	LOGIN	Alta	Alto
4	Robo de Datos	PAGO	Media	Critico
5	Escalada de Priv.	PANEL ADMIN	Media	Critico
6	CSRF	Login / PAGO	Media	Alto
7	Secuestro de Sesión	Autenticación	Media	Alto
8	DOS	Servidor Web	Media	Alto
9	Phishing / Ingeniería Social	Usuarios	Alta	Alto
10	Fuga por Error	BD / Servidor	Media	Alto

E.3 — Vulnerabilidades del Sistema

N°	Vulnerabilidad	Categoría OWASP	Corrección propuesta
1	Consultas sin parámetros	A03 inyección	Usar prepared statement
2	Falta cod. de salida	A03	CSP
3	Transmisión sin máscara	A02 Fallas criptográficas	bcrypt
4	Tráfico sin cifrar	A02	Forzar HTTPS/TLS 1.2+
5	Control de acceso	A01 Privilegios control de acc.	Aut. del servidor
6	sin protección anti-CSRF	A04 configuración	Tokens anti-CSRF
7	config insegura	A05 conf. de seg.	Hardening del servidor
8	Componentes desact.	A06 Componentes vul.	Gestión de parches
9	No hay límite de login	A07 Fallas de autenti.	Rate limit + bloqueo
10	No hay monitoreo	A09 Fallas de autenti.	Logins + alertas

E.4 — Posibles Atacantes

N°	Tipo de atacante	Motivación	Vector de ataque probable
1	Cibercriminal	Lucro económico	SQLi, malware
2	Empleado interno	Lucro	Abuso de privilegios
3	Hacktivista	ideología	DOS
4	Script kiddie	notoriedad	Exploits
5	Competencia/espía	ventaja comercial	Robo de datos

E.5 — Controles de Mitigación

N°	Amenaza / Vulnerabilidad	Control de mitigación	Tipo (prev/det/corr)
1	inyección SQL	prepared statement	preventivo
2	dependencias débiles	hashing bcrypt	prev
3	Fuerza Bruta	CAPTCHA, rate limiting	prev/detec.
4	datos de tránsito	Cifrado TLS/HTTPS	prev
5	Escalada de privilegios	Control de acceso por roles	prev.
6	XSS	Codificación de salida	prev.
7	Actividad	IDS/IPS	detectivo
8	Bugfix	Plan de respuesta	correctivo

F

CASOS DE USO SEGUROS

Complete las 3 plantillas con información técnica detallada

~25 min

Caso de Uso Seguro N° 1 — Inicio de Sesión Seguro

Actor(es)	USUARIO REGISTRADO
Objetivo	VERIFICAR IDENTIDAD DEL USUARIO
Precondición	Cuenta REGISTRADA y ACTIVA, Conexión cifrada HTTPS
Flujo principal	1. USUARIO SE LOGEA 2. SISTEMA VALIDA INGRESO 3. BUSCA EN EL HASH 4. VERIFICA SEGUNDO FACTOR 5. CON CONTE ACESSO FACTOR
Flujo alternativo	CHALLENGES INCONGRUOS
Riesgo asociado	FUERZA BRUTA, INYECCION SQL
Medida de protección	MASHING CON SAL, MFA/TLS, TOKENS

Caso de Uso Seguro N° 2 — Registro de Nuevo Usuario

Actor(es)	VISITANTE y SISTEMA
Objetivo	CREAR UNA NUEVA CUENTA DE FORMA SEGURA
Precondición	CONEXION HTTPS, CONTEO REGISTRADO
Flujo principal	1. EL USUARIO INGRESA SUS DATOS 2. SISTEMA VALIDA FORMATO 3. VERIFICA EMAIL 4. CALCULA HASH
Flujo alternativo	DATOS INVALIDO -> MENSAJE DE ERROR MENSAJE GENÉRICO
Riesgo asociado	INYECCION SQL, XSS, REGISTRO AUTOMATIZADO POR BOTS
Medida de protección	VALIDACION POR LISTA BLANCA, CAPTCHA, VERIFICACION DE CONTEO Y CONFIGURACION DE SSL/TLS

Caso de Uso Seguro N° 3 — Procesamiento de Pago

Actor(es)	CLIENTE AUTENTICADO, PASARELA DE PAGO, SISTEMA
Objetivo	PROCESAR PAGO CON TRANSACCION DE FORMA SEGURA
Flujo principal	1. CLIENTE CONFIRMA COMPRA 2. PASARELA AUTORIZA TRANSACCION 3. EL SISTEMA RECIBE TOKEN
Riesgo asociado	ROBO DE DATOS DE TARJETA, MITM, FRAUDE
Medida de protección	CUMPLIMIENTO PCI-DSS, TOKENIZACION, TLS INYECCION DE FRAUDE

G

PLAN DE DESARROLLO SEGURO

Escriba mínimo 3 actividades de seguridad por cada fase del SDLC

~10 min

Fase SDLC	Actividades de seguridad (mínimo 3 por fase)
Requerimientos ¿Qué preguntas de seguridad se deben hacer?	1) IDENTIFICAR REQUISITOS DE SEGURIDAD (PCI-DSS) 2) ANÁLISIS DE RIESGOS Y CLASIFICACIÓN DE ACTIVOS 3) DEFINICIÓN DE CASOS DE ABUSO Y CRITERIOS DE ACEPTACIÓN
Diseño ¿Qué patrones y controles arquitectónicos aplicar?	1) MANEJO DE AMENAZAS (STRIDE) 2) ARQUITECTURA SEGURA 3) DISEÑO DE AUTENTICACIÓN
Desarrollo ¿Qué buenas prácticas de codificación segura seguir?	1) CODIFICACIÓN SEGURA 2) GESTIÓN SEGURA DE SECRETOS 3) REVISIÓN DE CÓDIGO
Pruebas ¿Qué tipos de pruebas de seguridad ejecutar?	1) PRUEBAS DE PENETRACIÓN 2) ANÁLISIS DINÁMICO (DAST) 3) PRUEBAS DE CASOS DE ABUSO Y SEGURIDAD
Despliegue ¿Qué configuraciones y controles aplicar en producción?	1) MANEJO DE SERVIDORES 2) HTTPS/TLS 3) SERVICIO Y GESTIÓN DE PÁRAMES
Monitoreo ¿Qué alertas e indicadores de seguridad implementar?	1) LOGGING CENTRALIZADO 2) ALERTAS Y CORRELACIÓN (SIEM) + (IDS/IPS...) 3) DETECCIÓN DE ANOMALÍAS Y PLAN DE RESPUESTA



MINI RETO DE PC — LOGIN SEGURO EN JAVA

Actividad práctica en computadora

20%

~30 min



Programa Java: Validación de Login

Desarrolle un programa de consola que valide el acceso de un usuario con los requisitos de seguridad indicados a continuación.

Requisitos del programa:

- 1 Solicitar nombre de usuario por consola (Scanner).
- 2 Solicitar contraseña por consola.
- 3 Verificar longitud mínima de 8 caracteres.
- 4 Verificar presencia de al menos un número.
- 5 Verificar presencia de al menos una mayúscula.

Mensajes requeridos:

- 6 Mostrar mensaje específico para cada validación que falle.
- 7 Si todo es válido: "Acceso concedido. Bienvenido, [usuario]"
- 8 Bloquear tras 3 intentos fallidos consecutivos.

H.1 — Código fuente (pegue la captura de pantalla)

```

1 package loginseguro;
2
3 import java.util.Scanner;
4
5 public class LoginSeguro {
6
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         final int MAX_INTENTOS = 3;
10        int intentos = 0;
11
12        while (intentos < MAX_INTENTOS) {
13            System.out.print("Ingrese nombre de usuario: ");
14            String usuario = sc.nextLine().trim();
15
16            System.out.print("Ingrese contraseña: ");
17            String password = sc.nextLine();
18
19            if (validarCredenciales(usuario, password)) {
20                System.out.println("Acceso concedido. Bienvenido, " + usuario);
21                return;
22            } else {
23                intentos++;
24                int restantes = MAX_INTENTOS - intentos;
25                if (restantes > 0) {
26                    System.out.println("Intento fallido. Intentos restantes: " + restantes);
27                    System.out.println();
28                }
29            }
30
31            System.out.println("Cuenta bloqueada temporalmente tras "
32                + MAX_INTENTOS + " intentos fallidos.");
33        }
34
35        public static boolean validarCredenciales(String usuario, String password) {
36            boolean valido = true;
37
38            if (usuario == null || usuario.isEmpty()) {
39                System.out.println("Error: el nombre de usuario es obligatorio.");
40                valido = false;
41            }
42
43            if (password.length() < 8) {
44                System.out.println("Error: la contraseña debe tener al menos 8 caracteres.");
45            }
46
47            if (!password.matches(".*[0-9].*")) {
48                System.out.println("Error: la contraseña debe contener al menos un número.");
49                valido = false;
50            }
51
52            if (!password.matches(".*[A-Z].*")) {
53                System.out.println("Error: la contraseña debe contener al menos una mayúscula.");
54                valido = false;
55            }
56
57            return valido;
58        }
59    }
60 }
    
```

H.2 — Captura de ejecución (muestre: éxito · contraseña corta · sin número · sin mayúscula)

```

run:
Ingrese nombre de usuario: JHONN
Ingrese contraseña: 34ff
Error: la contraseña debe tener al menos 8 caracteres.
Error: la contraseña debe contener al menos una mayúscula.
Intento fallido. Intentos restantes: 2

Ingrese nombre de usuario: Jhonn
Ingrese contraseña: Camila1990
Acceso concedido. Bienvenido, Jhonn
BUILD SUCCESSFUL (total time: 25 seconds)
    
```


H.3 — Explique 3 líneas clave de su código

Nº	Fragmento de código	¿Qué hace y por qué es importante para la seguridad?
1	<code>if(password.length() < 8)</code>	EXIGE UNA LONGITUD MÍNIMA DE 8 CARÁCTERES, LAS CLAVES CORTAS SE ROMPEN FACILMENTE, NO SE FUGA LA INFORMACIÓN.
2	<code>password.matches("[0-9].*")</code>	OBLIGA A INCLUIR UN DÍGITO, AUMENTA LA COMPLEJIDAD Y EL ESFUERZO DE SUSQUER.
3	<code>while(intentos < MAX_INTENTOS)</code> {	LIMITA A LOS 3 INTENTOS Y BLOQUEA LA CUENTA; MITIGA DIRECTAMENTE LOS ATAQUES DE FUGA DE INFORMACIÓN AUTOMATIZADOS.



RÚBRICA DE EVALUACIÓN

Criterios de calificación — 20 puntos totales

20 pts
total

Sec.	Criterio	Indicadores de desempeño	Pts.
A	Completar la frase	Completa correctamente $\geq 16/20$ frases con el término técnico preciso.	3
B	Relacionar conceptos	Relaciona correctamente $\geq 16/20$ pares conceptuales.	3
C	Completar el código	Completa $\geq 12/15$ ejercicios con la instrucción Java correcta y funcional.	6
D	Análisis de código	Identifica el problema, explica el riesgo y propone corrección válida en $\geq 8/10$ fragmentos.	4
E	Modelado de amenazas	Completa las 5 tablas (activos, amenazas, vulnerabilidades, atacantes, controles) con coherencia técnica.	2
F	Casos de uso	3 casos de uso con todos los campos completos y técnicamente correctos.	2
H	Mini reto PC	Presenta código funcional, captura de ejecución y explica correctamente las 3 líneas clave.	20*
Σ	Nota mínima aprobatoria: 10.5 / 20 · Sección H se evalúa de forma independiente (20%)		20

Firma del docente: _____

Fecha de entrega: 02/06/2026